



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №3
на тему «Діаграма розгортання. Діаграма компонентів. Діаграма
взаємодій та послідовностей.»

Виконала:
Студентка групи ІА-22
Патрик Вікторія

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості.....	3
1. Діаграма розгортання (Deployment Diagram)	3
2. Діаграма компонентів (Component Diagram)	3
3. Діаграма взаємодій (Interaction Diagram).....	3
4. Діаграма послідовностей (Sequence Diagram).....	3
Тема	4
Реалізація частини функціональності системи	4
Посилання на проект на GitHub	5
Діаграма розгортання	6
Діаграма компонентів	8
Діаграма взаємодій та послідовностей.....	10

Діаграма розгортання. Діаграма компонентів. Діаграма взаємодій та послідовностей.

Мета: проаналізувати тему та створити діаграму розгортання, діаграму компонентів та діаграму взаємодій та послідовностей.

Хід роботи:

Короткі теоретичні відомості:

1. Діаграма розгортання (Deployment Diagram)

Відображає фізичне розміщення програмних компонентів на апаратних ресурсах, демонструючи, як елементи системи розподілені на сервери, комп'ютери або інші пристрої. Містить вузли (Nodes), які представляють обладнання або середовище виконання, та артефакти (Artifacts) — фізичні компоненти коду чи інших ресурсів.

2. Діаграма компонентів (Component Diagram)

Описує структуру системи на рівні компонентів (модулів) і їх зв'язків. Використовується для представлення архітектури системи, показуючи, як різні програмні компоненти взаємодіють між собою, та їх залежності. Компоненти можуть бути класами, бібліотеками, файлами або модулями.

3. Діаграма взаємодій (Interaction Diagram)

Включає діаграми послідовностей і кооперації, які відображають, як об'єкти взаємодіють між собою для виконання певної задачі. Діаграма взаємодій показує повідомлення, якими обмінюються об'єкти, та їхній порядок.

4. Діаграма послідовностей (Sequence Diagram)

Показує порядок взаємодій між об'єктами в часі, відображаючи повідомлення, що передаються між ними, та їх часову послідовність. Використовується для моделювання динамічних аспектів системи і визначення логіки сценаріїв або випадків використання.

Тобто Діаграма взаємодій та послідовностей дозволяє глибше зрозуміти, як об'єкти взаємодіють у межах конкретних сценаріїв, і водночас отримати загальне уявлення про зв'язки між об'єктами в системі.

Тема:

..3 Текстовий редактор (strategy, command, observer, template method, flyweight, SOA)

Текстовий редактор повинен вміти розпізнавати текстові файли в будь-якій кодуванні, мати розширені функції редагування: макроси, сніппети, підказки, закладки, перехід на рядок / сторінку, підсвічування синтаксису (для однієї мови програмування або розмітки на розсуд студента).

Реалізація частини функціональності системи:

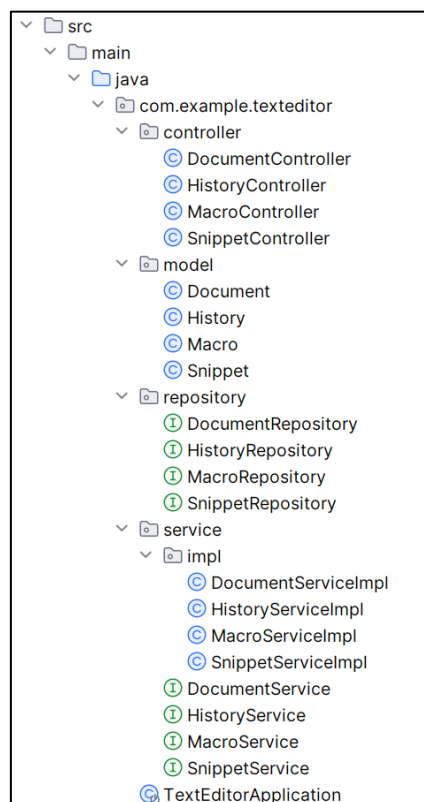


Рисунок 1. Структура проекту

В моєму проекті **TextEditor** структура побудована з використанням архітектури багаторівневого розподілу:

1. **controller** – містить контролери, які обробляють запити від користувачів:
 - DocumentController, HistoryController, MacroController, SnippetController – контролери для управління відповідними сутностями (документами, історією, макросами, фрагментами коду).
2. **model** – зберігає моделі даних:

- Document, History, Macro, Snippet – сутності, що представляють різні частини текстового редактора (наприклад, документ, історія змін, макроси, фрагменти).
3. **repository** – включає репозиторії для доступу до бази даних:
 - DocumentRepository, HistoryRepository, MacroRepository, SnippetRepository – інтерфейси для збереження та отримання даних для кожної сутності.
 4. **service** – містить бізнес-логіку проекту:
 - Інтерфейси DocumentService, HistoryService, MacroService, SnippetService та їх реалізації у папці **impl** (DocumentServiceImpl, HistoryServiceImpl, MacroServiceImpl, SnippetServiceImpl), що виконують основні операції з відповідними сутностями.
 5. **TextEditorApplication** – головний клас, який запускає Spring Boot додаток.

Ця структура дозволяє легко керувати різними компонентами програми, розділяючи логіку контролерів, бізнес-логіку, доступ до даних та моделі.

Посилання на проект на GitHub:

<https://github.com/vvvvvvvvvviktoriia/TextEditor>

Діаграма розгортання:

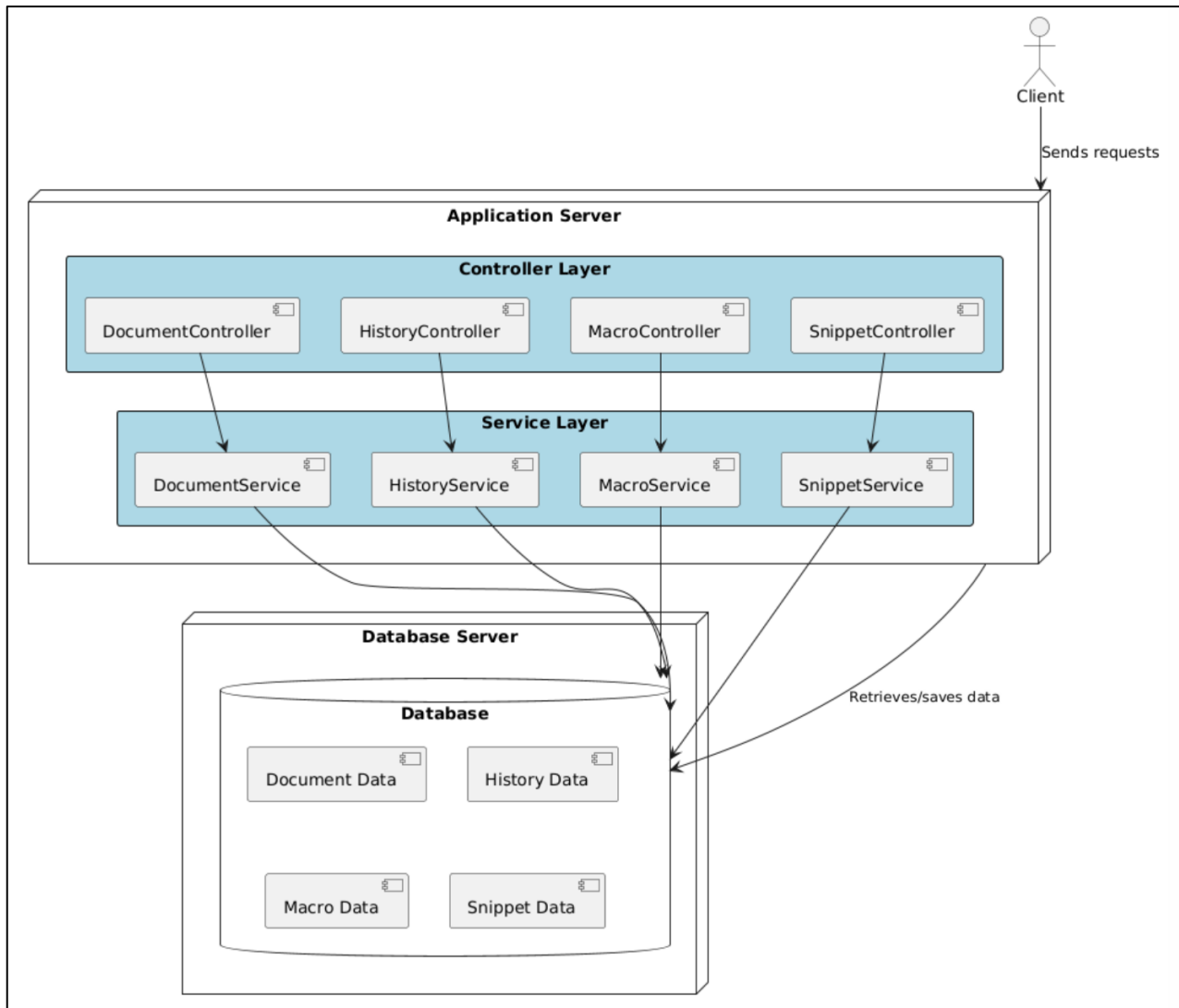


Рисунок 2. Діаграма розгортання.

Ця діаграма розгортання, яку ми бачимо на Рисунку 2 зображує взаємодію клієнта з серверною частиною та базою даних у системі текстового редактора, який має розширені функції, такі як управління документами, історією змін, макросами та фрагментами коду.

1. **Клієнт:** Користувач надсилає запити до системи через інтерфейс клієнта.
2. **Сервер застосунків (Application Server):** Цей сервер обробляє запити від клієнта та виконує основну бізнес-логіку. Він складається з:
 - **Контролерного шару (Controller Layer),** який містить контролери для кожної основної функціональної області:
 - DocumentController для управління документами.

- HistoryController для відстеження та збереження історії змін.
 - MacroController для роботи з макросами.
 - SnippetController для управління фрагментами коду.
- **Сервісного шару (Service Layer)**, що реалізує бізнес-логіку, необхідну для обробки запитів, які отримує контролерний шар:
- DocumentService для обробки документів.
 - HistoryService для роботи з історією змін.
 - MacroService для роботи з макросами.
 - SnippetService для обробки фрагментів коду.
3. **Сервер бази даних (Database Server)**: Використовується для зберігання всіх даних, необхідних для функціонування системи. Він містить:
- **Базу даних (Database)** з кількома таблицями або структурами для зберігання:
- Document Data — дані документів.
 - History Data — дані історії змін.
 - Macro Data — дані макросів.
 - Snippet Data — дані фрагментів коду.

Зв'язки:

- Клієнт надсилає запити до серверу застосунків, де контролери обробляють ці запити, викликаючи відповідні сервіси.
- Сервіси серверу застосунків з'єднуються з сервером бази даних для отримання чи збереження необхідної інформації, взаємодіючи з відповідними таблицями бази даних.

Ця структура забезпечує розділення обов'язків між різними компонентами системи, дозволяючи контролерам обробляти запити, сервісам — реалізовувати бізнес-логіку, а базі даних — зберігати та керувати даними.

Діаграма компонентів:

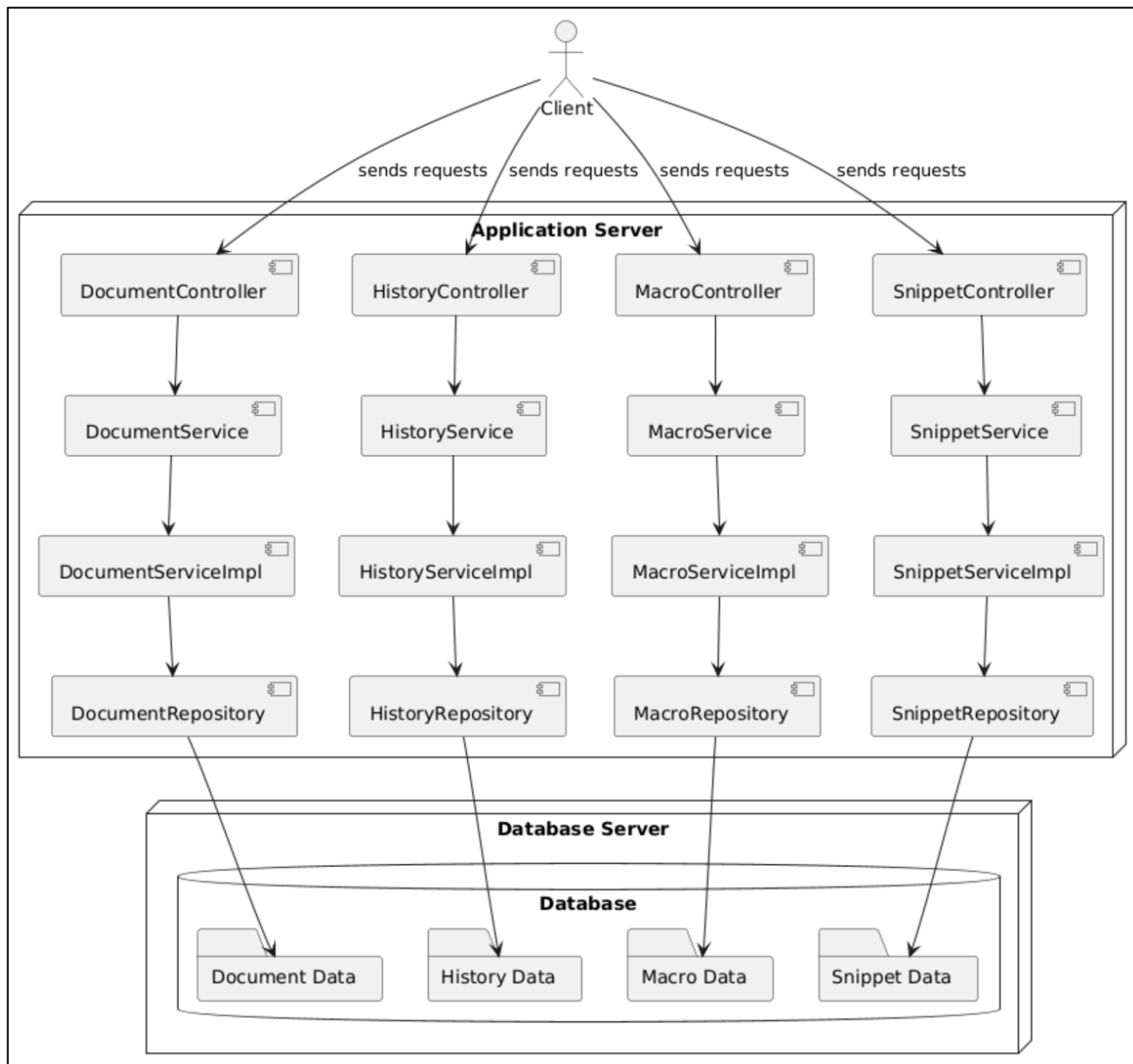


Рисунок 3. Діаграма компонентів.

Ця діаграма компонентів, яку ми бачимо на Рисунку 3 зображує архітектуру текстового редактора, що складається з різних компонентів, організованих за функціональними рівнями на сервері застосунків та сервері бази даних.

1. **Клієнт:** Користувач взаємодіє з текстовим редактором, надсилаючи запити для виконання різних функцій, таких як управління документами, макросами, історією та фрагментами коду.
2. **Сервер застосунків (Application Server):** Містить логіку обробки запитів клієнта та складається з трьох рівнів:
 - **Контролери** (DocumentController, HistoryController, MacroController, SnippetController): Приймають запити від клієнта та перенаправляють їх на відповідні сервіси для обробки.

- **Сервіси** (DocumentService, HistoryService, MacroService, SnippetService): Реалізують бізнес-логіку для кожного типу операцій у текстовому редакторі.
 - **Реалізації сервісів** (DocumentServiceImpl, HistoryServiceImpl, MacroServiceImpl, SnippetServiceImpl): Містять фактичну реалізацію методів сервісів і взаємодіють з репозиторіями для доступу до даних.
3. **Репозиторії (Repositories):** Кожен сервіс використовує відповідний репозиторій (DocumentRepository, HistoryRepository, MacroRepository, SnippetRepository) для роботи з даними. Репозиторії є інтерфейсом для взаємодії з базою даних.
4. **Сервер бази даних (Database Server):** Зберігає всі дані, що необхідні для роботи текстового редактора:
- **Document Data:** Дані, пов'язані з документами.
 - **History Data:** Дані історії змін.
 - **Macro Data:** Дані макросів.
 - **Snippet Data:** Дані фрагментів коду.

Взаємодії:

- Клієнт надсилає запити до контролерів, які потім викликають відповідні сервіси.
- Сервіси взаємодіють зі своїми реалізаціями, які звертаються до репозиторіїв для доступу до бази даних.
- Репозиторії обробляють запити до бази даних, зберігаючи та отримуючи дані для кожного типу об'єктів.

Ця діаграма забезпечує чітку структуру, де кожен компонент має конкретну роль, що полегшує розширення, тестування та підтримку системи текстового редактора.

Діаграма взаємодій та послідовностей:

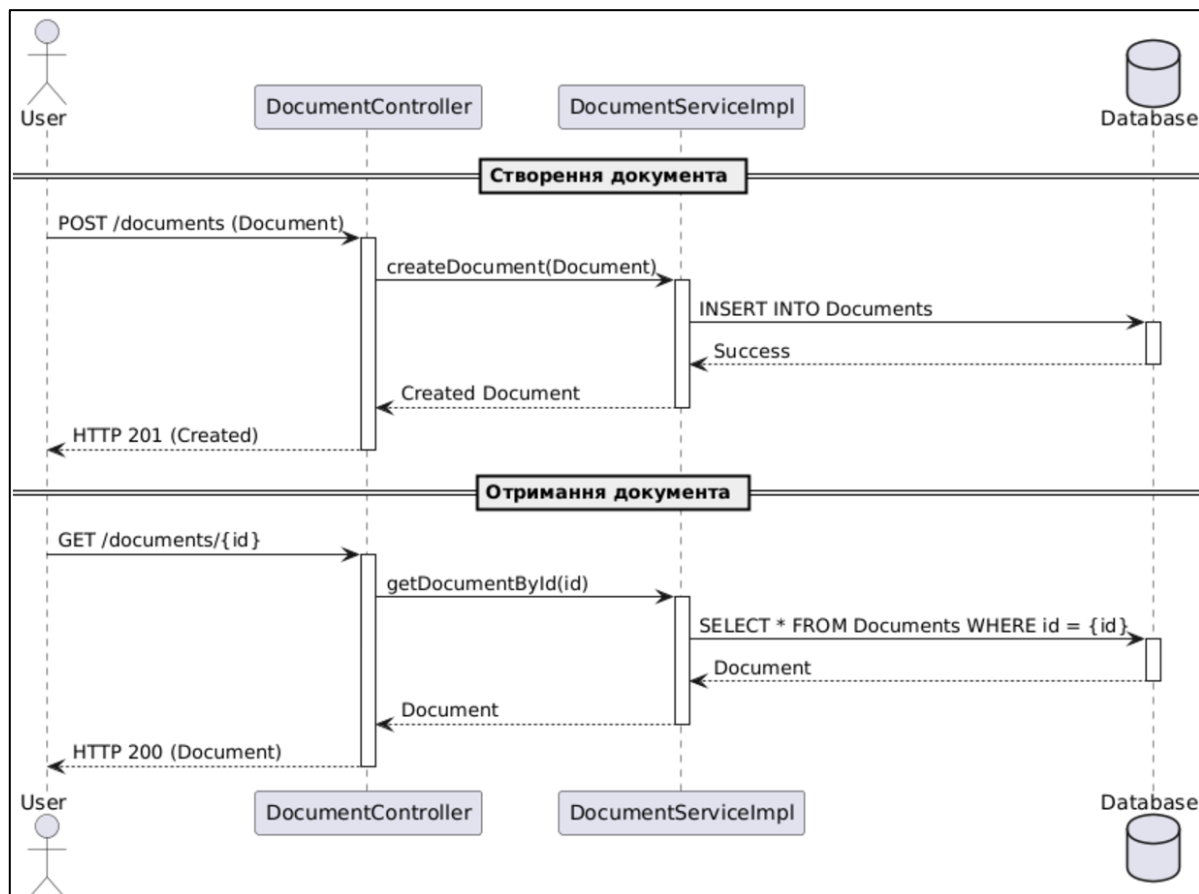


Рисунок 4. Діаграма взаємодій та послідовностей.

Ця діаграма, яку ми бачимо на Рисунку 4 ілюструє послідовність взаємодій між різними компонентами проєкту текстового редактора під час виконання основних операцій: створення документа, доступу до історії документа, виконання макросу та створення сніпета. Кожен з цих процесів включає використання відповідних контролерів, сервісів та репозиторіїв, які взаємодіють між собою для досягнення мети.

1. Створення документа

- **Користувач** ініціює процес створення нового документа, відправляючи запит до DocumentController.
- DocumentController викликає метод createDocument у DocumentService, передаючи необхідні дані документа.
- DocumentService звертається до DocumentRepository з метою збереження нового документа в базі даних.
- Після успішного збереження DocumentRepository повертає підтвердження в DocumentService, який передає це підтвердження назад у DocumentController.
- **Користувач** отримує повідомлення про успішне створення документа.

2. Доступ до історії документа

- **Користувач** надсилає запит до `HistoryController` для перегляду історії певного документа.
- `HistoryController` викликає метод `getHistory` у `HistoryService`, передаючи ідентифікатор документа.
- `HistoryService` звертається до `HistoryRepository` для отримання історії змін документа.
- `HistoryRepository` повертає знайдену історію до `HistoryService`, який передає її назад у `HistoryController`.
- **Користувач** отримує історію документа для перегляду.

3. Виконання макросу

- **Користувач** виконує запит на виконання макросу через `MacroController`.
- `MacroController` викликає метод `executeMacro` у `MacroService`, передаючи ідентифікатор макросу.
- `MacroService` звертається до `MacroRepository`, щоб знайти відповідний макрос за ідентифікатором.
- Після отримання даних макросу `MacroService` виконує необхідні операції та повертає підтвердження у `MacroController`.
- **Користувач** отримує підтвердження про успішне виконання макросу.

4. Створення сніпета

- **Користувач** надсилає запит до `SnippetController` для створення нового сніпета.
- `SnippetController` викликає метод `createSnippet` у `SnippetService`, передаючи дані сніпета.
- `SnippetService` передає ці дані до `SnippetRepository`, щоб зберегти новий сніпет у базі даних.
- Після успішного збереження `SnippetRepository` повертає підтвердження в `SnippetService`, який передає це підтвердження назад у `SnippetController`.
- **Користувач** отримує повідомлення про успішне створення сніпета.

Ця діаграма показує, як контролери отримують запити від користувача та передають їх у відповідні сервіси. Сервіси, в свою чергу, взаємодіють з репозиторіями для доступу до бази даних або виконання логіки обробки даних. Це забезпечує чітке розділення обов'язків між компонентами, що сприяє кращій структуризації коду, полегшує обслуговування та підтримку проєкту.

Висновок: у ході виконання лабораторної роботи було проведено аналіз архітектури системи та створено кілька UML-діаграм, які дозволили детально зрозуміти її структуру та функціонування. Діаграма розгортання допомогла візуалізувати фізичне розташування компонентів системи та їх взаємодію в реальному середовищі, що сприяло визначенню ефективної конфігурації інфраструктури. Діаграма компонентів відобразила логічну структуру програмного забезпечення, зв'язки між його модулями та ключові інтерфейси, що забезпечило розуміння взаємозалежностей між різними частинами системи. Діаграма взаємодій і послідовностей продемонструвала динаміку виконання запитів, порядок обробки операцій та зв'язок між користувачем, системою і базою даних, що дало змогу побудувати чітку модель поведінки програми. Усі ці діаграми стали важливим інструментом для проєктування, аналізу та документування системи, допомогли виявити слабкі місця архітектури й оптимізувати її для подальшої реалізації.